Teaching Statement

Stanisław Szufa
University of Geneva

Teaching Philosophy: Learning by Measuring

The most useful thing I learned during my PhD, I had to learn on my own. My advisor's work was theoretical, and when I wanted to know how voting rules actually behave, nobody around me could tell me how to design a numerical experiment, which data to use, or how to tell a real finding from an artifact of the generator. Working this out became my thesis, and then a tutorial, a community guide, and two software libraries, so that others would not have to start from scratch. This experience shapes how I want to teach. I want students to *measure* the objects they study, not only to prove things about them: a student who has computed an optimal consensus ranking for a real election and watched an integer-programming solver stall on it understands NP-hardness in a way that no reduction on a blackboard can give. The evidence on active learning in STEM points the same way (Freeman et al., *PNAS*, 2014).

When I design my own courses, this will mean three things:

- *Every abstraction comes with an instance.* I want to introduce each definition together with a dataset on which students can try it in the same session. In my field this is easy: a city's participatory budget is a real election with real stakes, and students can load one from Pabulib, run three voting rules on it, and argue about which outcome is fairer before they have seen a single axiom.
- *Students should discover, not only verify.* Standard exercises build the foundations, but the course projects I want to set have answers that I do not know in advance. "Which random model of voters best approximates the preferences of Warsaw's residents?" is a question a second-year student can attack with the tools from my research, and, as the student projects I have co-supervised show, the answers can be publishable.
- *Tools are part of the curriculum.* Writing clean, reproducible experimental code is a skill that most students are never taught. I intend to teach it explicitly, with open-source libraries such as `prefsampling` and `mapof` as real codebases that students read, test, and extend instead of writing throwaway scripts.

Teaching Experience

As a Teaching and Research Assistant at Jagiellonian University (2018–2021), I taught the exercise sessions of three core bachelor courses in computer science: *Introduction to Programming*, *Logic and Set Theory*, and *Linear Algebra*. These were courses designed by others, with large first- and second-year cohorts and a wide spread of preparation, and they taught me to explain the same idea in several ways and to notice early which students are falling behind. They also cover the material I would most like to teach at the bachelor level at EPFL: programming, discrete mathematics, and algorithms are the foundation on which everything in my research stands.

My own research I have so far taught only outside the classroom, to my research community. I gave a tutorial on numerical experiments in computational social choice at three of the major AI conferences, AAAI, AAMAS, and IJCAI, and I co-authored the *Guide to Numerical Experiments on Elections* (IJCAI-24), written as teaching material: it explains what goes wrong in experiments and how to avoid it. Other groups now use `prefsampling` and `mapof` in their research and in student projects. Explaining voting rules to city officials and residents through *Science for Democracy* has been the most demanding teaching I have done: the audience has no patience for notation and every question is about a real decision.

Mentoring and Supervision

I have co-supervised more than ten bachelor's and master's students, mostly at AGH University of Kraków, within an ERC-funded project, and at LAMSADE in Paris. Several of these projects grew into publications at AAAI, AAMAS, ECAI, and NeurIPS with the student as a coauthor. My approach follows how I was mentored, and how I wish I had been:

- *Start small and concrete.* A student's first project is a well-defined experimental question with a dataset and a tool already in hand, so that the first result comes within weeks.
- *Then hand over an open problem.* Once a student has shipped something, I give them a question whose answer I do not know, and I expect them to disagree with me. The best paper award I received at AAMAS-23 was for a paper written by three PhD students without a senior coauthor, and I want my own students to have that experience.
- *Teach the craft, not only the result.* I review students' code and experimental design as carefully as their proofs, and I ask them to present their work early and often: clear writing and presenting decide whether good work is noticed.

As President of the PhD Students' Association at Jagiellonian University and as a representative in the Polish national association of doctoral candidates, I spent two years working on funding, supervision rules, and representation of doctoral students. I learned how much a student's success depends on things outside the research itself, and I would attend to them in a group of my own.

Teaching at EPFL

Existing courses. I am prepared to teach core courses of the computer science bachelor, in particular introductory programming, discrete mathematics (as in *Advanced Information, Computation, Communication I*, CS-101), *Algorithms I* (CS-250), and *Theory of Computation* (CS-251). At the master level, I am prepared to teach game theory, including *Algorithmic Game Theory* (MGT-435), from equilibria and mechanism design to the social-choice topics closest to my research.

New courses. I would like to develop two courses that complement *Technologies for Democratic Society* (CS-234), which studies the systems behind democratic processes, and *Algorithmic Game Theory*, which studies equilibria and mechanism design:

- *Algorithms for Collective Decisions* (master level). Voting, committee elections, participatory budgeting, and fair division, treated as algorithmic and data problems: every rule comes with real data and every axiom with an experiment. By the end, students should be able to choose an aggregation rule for a concrete decision, compute it efficiently, test its properties on real and synthetic data, and explain the choice to the people it affects. The project is a full study of a real participatory process, from data to recommendation.
- *AI and Democracy* (master seminar). How language models change collective decision-making: AI-mediated deliberation, LLM proxies, representation in generated text, and the risks of each. The seminar would combine recent papers with hands-on simulation, follow on naturally from CS-234, and bring together students of computer science, data science, and digital humanities.

Growing as a teacher. As my classroom experience is limited to exercise sessions, I would make full use of the induction seminar and pedagogical consultations offered by EPFL's Teaching Support Center (CAPE), and in every course I would ask students for feedback mid-semester, while there is still time to act on it.

Beyond the classroom. I would bring my work with Science for Democracy to EPFL through student projects with Swiss municipalities that run participatory budgets. Teaching residents why a proportional rule funded one project and not another is, to me, the same job as teaching students why an algorithm is correct.